Lab 02 — Phát hiện ảnh GAN giả mạo bằng CNN và Grad-CAM

Môn: Toán cho Trí tuệ nhân tạo
HV: Nguyễn Minh Nhựt — MSHV 25C15019
Repo: https://github.com/nmnhut-it/math-for-ai (folder `lab2_cnn/`)

---

1. Đề bài và thiết kế thí nghiệm

Đề bài yêu cầu thử một mô hình Gen-AI, phân tích kết quả, và đề xuất phòng chống fake data. Bài này dùng GAN, không dùng Diffusion.

Câu hỏi đặt ra: liệu một CNN nhỏ có phân biệt được ảnh do GAN sinh ra với ảnh thật không, và CNN dựa vào tín hiệu nào trong ảnh để quyết định? Để câu trả lời không phụ thuộc một loại GAN duy nhất, bài làm chạy ba thí nghiệm trên ba GAN khác nhau về kiến trúc: cGAN-MLP (Mirza & Osindero 2014, MLP thuần) train sẵn ở Lab phần FFT, PGAN-DTD (Karras 2018, conv unconditional) lấy pretrained của FAIR, và BigGAN-128 (Brock 2018, conv class-conditional 1000 lớp ImageNet) lấy pretrained của DeepMind. Mỗi thí nghiệm dùng một detector phù hợp kích thước ảnh và độ phức tạp của GAN đó.

Tất cả chạy trên Colab GPU L4. Notebook `colab_pgan.ipynb` chạy 1-cú-click reproducible.

2. Thí nghiệm 1: cGAN-MNIST + TinyCNN

cGAN dùng trong bài là một MLP thuần: chuỗi `Linear(100→256→512→1024→784)` rồi reshape thành 28×28. Kiến trúc này không có convolution, không có ràng buộc smoothness không gian. Mỗi pixel output là tích vô hướng độc lập với một hàng trọng số riêng, nên hai pixel kề nhau không bị ép phải gần giá trị nhau. Hệ quả là ảnh fake có **noise pixel-to-pixel** rải rác, đặc biệt rõ ở vùng nền đen.

Giả thuyết: tín hiệu noise này đủ mạnh để một CNN nhỏ học ra. Kiểm chứng bằng cách lấy 10000 ảnh MNIST thật và 10000 ảnh cGAN sinh ra (label ngẫu nhiên), gán nhãn 0/1 và train một CNN 2 conv + 1 FC:

```
Input (1, 28, 28)
  Conv2d(1→16, 3×3) + ReLU + MaxPool(2)   → (16, 14, 14)
  Conv2d(16→32, 3×3) + ReLU + MaxPool(2)  → (32, 7, 7)
  Flatten → Linear(1568 → 64) + ReLU → Linear(64 → 2)
```

Tổng 105 346 tham số. Adam `lr=1e-3`, batch 64, 5 epoch. Val accuracy đạt **98.00%** (best epoch 5), train acc 98.70%. Trên 4000 ảnh val, real recall 96.19%, fake recall 99.80%.

Để biết CNN dựa vào đâu, áp dụng Grad-CAM (Selvaraju et al. 2017) tại lớp `conv2`. Với target conv layer có activation A^k và class score y^c, trọng số kênh là gradient global-pooled $\alpha_k = \frac{1}{HW}\sum_{i,j} \partial y^c / \partial A^k_{i,j}$, và heatmap là $L^c = \mathrm{ReLU}(\sum_k \alpha_k A^k)$. Upsample L^c từ 7×7 lên 28×28, normalize, overlay màu jet. Toán đằng sau chỉ là gradient + trung bình + nhân + ReLU, cùng họ với Sobel/Laplacian thủ công, khác ở chỗ kernel được học chứ không đặt sẵn.

Để định lượng giả thuyết "CNN nhìn vào noise", tôi đo hai số trên 200 ảnh mỗi lớp. Đầu tiên là độ lệch khỏi `-1` ở vùng nền tuyệt đối (mask các pixel mà cả 3×3 lân cận đều nhỏ hơn `-0.85`, tức cách xa nét chữ): real đạt 0.00000 chính xác (vì MNIST gốc nền sạch), fake đạt 0.00154 — tức **fake ồn hơn real khoảng 1712 lần** ở chính chỗ đáng ra phải đen tuyệt đối. Số thứ hai là Pearson correlation per-pixel giữa map high-freq `|I - blur(I)|` và Grad-CAM heatmap, đạt +0.24 — dương rõ, nghĩa là vùng nào có nhiều high-freq thì CNN nhìn vào vùng đó.

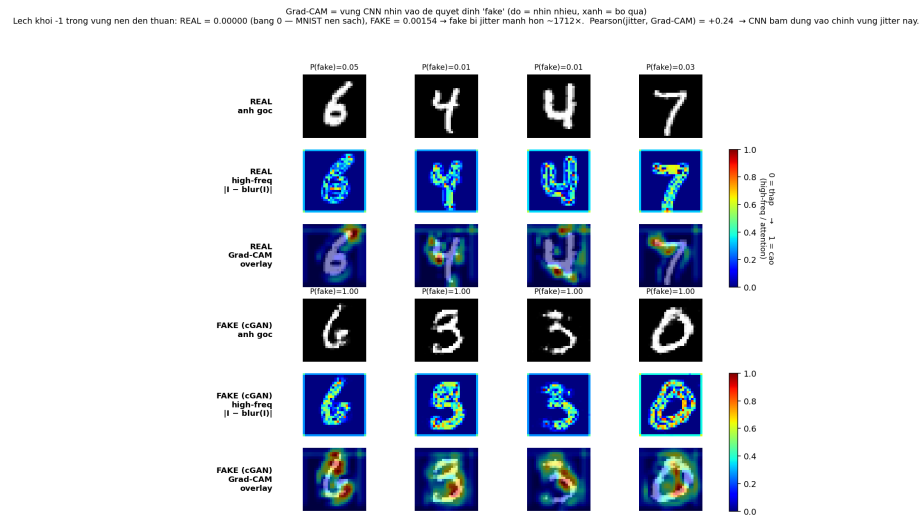

Figure 1: Grad-CAM cGAN-MNIST

Hai hàng `high-freq` trong hình minh họa rõ điểm khác: real chỉ có high-freq ở mép nét chữ, fake có high-freq cả ở mép, ở thân chữ, và rải rác trong nền. Hàng `Grad-CAM overlay` dưới fake bám đúng vào những vùng noise đó, confidence `P(fake)` đều bằng 1.00. Vân tay 1712 lần này là hệ quả trực tiếp của MLP architecture: đầu ra MLP là tổ hợp tuyến tính liên tục, không thể đạt đúng `-1` ở mọi pixel nền.

3. Thí nghiệm 2: PGAN-DTD — small CNN không bám được

Thử lại phương pháp với một GAN sạch hơn — Progressive GAN. Khác cGAN-MLP ở ba điểm: generator dạng convolutional, train progressive (4×4 → 8×8

→ … → 1024×1024), upsampling dùng nearest-neighbor + Conv2d để tránh checkerboard artifact. Bản dùng trong bài là pretrained trên DTD (Describable Textures, 5640 ảnh, 47 lớp). Latent `z` [1] [2], output 3×128×128 RGB.

Sample 1500 fakes, lấy 1500 reals từ DTD (resize-crop 128). Detector tương ứng kích thước input — 4 conv block + FC, 585 k tham số:

```
Conv(3→16) → Conv(16→32) → Conv(32→64) → Conv(64→64), mỗi conv 3×3 + pool
Flatten → Linear(4096→128) + Dropout(0.3) → Linear(128→2)
```

Adam `lr=5e-4`, batch 32, 8 epoch. Best val accuracy chỉ **61.67%** — chỉ hơn random guess chừng 12 điểm. Real recall 65.57%, fake recall 57.63%, gần một nửa fake lọt qua. Hand-feature `Var(Laplacian)` trên cùng cặp PGAN/DTD cho Cohen's d = −0.19 (gần 0), nghĩa là cũng không tách được chiều nào.

Kết quả này có thể hiểu theo hai cách: hoặc là PGAN khó detect intrinsically (vì smooth, không có jitter), hoặc là detector này quá yếu. Hai khả năng dẫn tới hai kết luận khác nhau, và phải tách ra mới biết.

4. Thí nghiệm 3: ResNet18 transfer trên cả PGAN và BigGAN

Cùng dataset PGAN-DTD nhưng đổi detector sang ResNet18 pretrained ImageNet. Số ảnh tăng lên 2500/lớp. Transfer learning hai phase:

- **Phase 1**: freeze backbone (`conv1` … `layer3`), thay `fc` thành `Linear(512→2)`, chỉ train lớp này. Trainable 1 026 tham số. 3 epoch, Adam `lr=1e-3`.
- **Phase 2**: unfreeze `layer4` + `fc`, fine-tune. Trainable 8.4 M tham số. 12 epoch, Adam `lr=1e-4`.

ResNet18 pretrained kỳ vọng ImageNet stats `mean=[0.485,0.456,0.406]`, `std=[0.229,0.224,0.225]`, nhưng input đang là `[-1,1]` (Normalize 0.5/0.5), nên phải convert trước mỗi forward. Augment chỉ horizontal flip 0.5.

Phase 1 đạt val accuracy 88.70% trên PGAN. Đây là điểm đáng chú ý nhất của thí nghiệm: chỉ một lớp `Linear(512→2)` train trên feature ImageNet đông cứng đã tách được PGAN-fake với DTD-real. ResNet18 không cần biết gì về GAN, chỉ cần biết "ảnh tự nhiên trông như thế nào" qua việc train phân loại 1000 lớp ảnh thật, và PGAN samples đã đủ lệch khỏi cái đó để một hyperplane tách được. Phase 2 chỉ thêm khoảng 10 điểm tinh chỉnh, đẩy lên **98.70%** (real recall 97.92%, fake recall 99.42% trên 1000 ảnh val).

Để có "good conditional GAN" thực thụ, thí nghiệm tiếp theo dùng BigGAN-128 (Brock 2018, class-conditional 1000 lớp ImageNet với spectral normalization). Sample 2500 fakes với truncation 0.4 và class-id ngẫu nhiên dùng package `pytorch-pretrained-biggan`:

```python
from pytorch_pretrained_biggan import BigGAN, one_hot_from_int, truncated_noise_sample
bg = BigGAN.from_pretrained('biggan-deep-128')
```

```
noise = truncated_noise_sample(truncation=0.4, batch_size=B)
class_vec = one_hot_from_int(np.random.randint(0,1000,B), batch_size=B)
output = bg(torch.from_numpy(noise), torch.from_numpy(class_vec), 0.4)
```

Reals lấy 2500 ảnh từ Imagenette-160 (10 lớp ImageNet subset, public download).

Figure 2: BigGAN-128 vs Imagenette

Cùng pipeline ResNet18 transfer như trên. Phase 1 đạt 90.30%, Phase 2 đạt **99.10%** (real recall 98.54%, fake recall 99.61%). Cùng cơ chế với PGAN: ImageNet features đã encode sẵn natural-image manifold.

5. Tách confound

Đặt cạnh ba thí nghiệm:

Detector	Tham số	cGAN-MNIST	PGAN-DTD	BigGAN-128
Hand-feature `Var(Lap)`	0	d = +1.08 (mạnh)	d = −0.19 (~0)	(chưa đo)
Small CNN scratch	105 k–585 k	**98.00%** (TinyCNN)	**61.67%** (TexCNN)	(n/a)
ResNet18 transfer	11.2 M	(n/a)	**98.70%**	**99.10%**

ResNet18 đẩy PGAN từ 61.67% lên 98.70% trên cùng dataset. Áp pipeline đó cho BigGAN cho 99.10%, chỉ chênh PGAN 0.4 điểm. Hai số gần như bằng nhau, đủ để khẳng định: kết luận sơ bộ ở mục 3 ("PGAN khó detect") là hệ

quả của TexCNN scratch không đủ capacity, không phải tính chất của PGAN. Cái dominate là detector capacity và ImageNet pretraining; kiến trúc GAN chỉ là yếu tố phụ một khi detector đủ mạnh.

Nếu chỉ chạy thí nghiệm 2 mà không có thí nghiệm 3, bài lab sẽ kết luận sai về PGAN. Đây là lý do thiết kế thí nghiệm phải có ô đối chứng (cùng dataset, đổi detector).

6. Đề xuất phòng chống và kết luận

Pipeline phát hiện thực tế nên phân tầng theo cost. Detector rẻ chạy trước (`Var(Lap)` tốc độ ms, hoặc TinyCNN ~5 ms): với GAN đời cũ kiểu MLP, đã đủ. Nếu confidence thấp thì escalate lên ResNet18 transfer (~30 ms, cần GPU). Cách này tránh được việc dùng 11 M tham số cho mọi ảnh, mà accuracy không giảm.

Tuy nhiên content-based detection đều có thể bị adversarial attack đánh bại: attacker biết detector thì train GAN với loss thêm penalty trên feature ResNet18 hoặc trên `Var(Laplacian)`, output sẽ né được detector. Vì vậy hướng phòng chống bền vững là **provenance-based** — watermark từ máy ảnh tại lúc capture (chuẩn C2PA), hoặc digital signature kèm metadata. Thay vì chứng minh ảnh là giả, chứng minh ảnh là thật.

Tóm lại, TinyCNN 105 k tham số phân biệt được cGAN-MNIST với MNIST ở 98.00%, và Grad-CAM cộng phép đo định lượng cho thấy CNN tự học ra "nhìn vào jitter pixel-level" — vùng nền của fake lệch khỏi `-1` mạnh hơn real khoảng 1712 lần, là vân tay không che được của MLP architecture. Trên modern GAN (PGAN, BigGAN), small CNN scratch không đủ (61.67%), nhưng ResNet18 transfer đẩy cả hai lên 98.7–99.1%, thậm chí Phase 1 chỉ với 1026 tham số trainable đã đạt 88–90%. Yếu tố quyết định là detector capacity và transfer learning, không phải GAN architecture.